

CoreOS: Distributed Transaction Manager (DTM)

Operating Systems Project Documentation

Developer: Abdullah Falak

Program: BSCS (5th Semester)

Environment: Kali Linux | C Language | POSIX Standard

1. Project Overview

The CoreOS Distributed Transaction Manager (DTM) is a simulated system-level application built in C. It demonstrates how modern operating systems manage concurrent tasks efficiently and safely. By simulating an automated transaction environment, the project showcases the lifecycle of process creation, job scheduling, resource allocation, and inter-process synchronization using the Linux kernel's core POSIX libraries.

2. System Architecture & Workflow

The project runs in a single, robust cycle that mimics how an OS handles a burst of concurrent jobs:

- **Initialization:** The Master process initializes POSIX Shared Memory to act as a global ledger. It also configures standard Signals to allow safe, immediate termination if a critical error occurs.
- **CPU Scheduling:** A batch of incoming transaction jobs is placed into a queue. The system sorts these jobs using the Shortest Job First (SJF) scheduling algorithm to optimize turnaround time.
- **Safety Verification:** Before any processes are spawned, the Banker's Algorithm assesses the resource requirements of the scheduled jobs. It verifies that executing these jobs will not lead the system into a Deadlock state.
- **Concurrent Execution:** The Master process forks multiple Child processes (Workers). Each Worker represents a transaction.
- **Synchronization:** As Workers execute concurrently, they use Mutexes and Semaphores to lock the shared ledger and restrict the number of active database

connections, preventing race conditions.

- **Termination & Cleanup:** The Master uses the wait mechanism to ensure all Workers complete their execution before safely destroying the shared memory and shutting down.

3. Integration of Core OS Concepts

This project seamlessly integrates 10 fundamental Operating System concepts into a single cohesive application:

1. Fork

The `fork()` system call is used by the Master process to create exact copies of itself (Child/Worker processes) to handle individual transactions concurrently.

2. Wait

The `wait()` system call forces the Master process to pause its own execution until all forked Child processes have completed their tasks, preventing the creation of 'zombie' processes.

3. Sleep

The `sleep()` function is utilized within the Worker processes to simulate the burst time (CPU execution time) required to process a complex transaction.

4. CPU Scheduling

A custom implementation of the Shortest Job First (SJF) algorithm sorts the incoming transaction queue based on burst time before execution begins.

5. Mutex

A binary semaphore (POSIX `sem_t`) is used as a Mutex to lock the shared ledger. When one Worker is deducting funds, the Mutex ensures no other Worker can read or write to the ledger simultaneously.

6. Semaphore

A counting semaphore is implemented to represent a limited pool of database connections (e.g., only 2 concurrent slots available). Workers must acquire a slot before proceeding.

7. Banker's Logic

The Banker's Algorithm is programmed into the `is_safe_state()` function. It evaluates the maximum resources each job might need against currently available resources.

8. Deadlock Avoidance

Relying on the Banker's logic, the system ensures it only proceeds with job execution if it guarantees a 'Safe State', completely avoiding deadlocks.

9. Msg Signal

The `<signal.h>` library is used to capture `SIGINT` (interrupt signals). If the Banker's algorithm detects an unsafe state, a signal is sent to safely destroy IPC resources and abort the program.

10. IPC (Inter-Process Communication)

POSIX Shared Memory (`shmget`, `shmat`) is established so the isolated Master and Worker processes can share and update a single struct containing the global ledger and synchronization locks.

4. Compilation & Execution Instructions

Because the project heavily utilizes the POSIX threads library for synchronization (semaphores and mutexes), it requires a specific compilation flag in Linux.

1. Compile the code using GCC:

```
gcc main.c -o core_project -pthread
```

2. Execute the compiled binary:

```
./core_project
```